

SysML v2 Fix Recipes

This file documents common syntactical and semantic errors generated by LLMs when writing SysML v2 code, and the recipes used to fix them.

1. Satisfy and Frame Keywords

Error: LLMs often write `satisfy requirement req;` or `frame concern c;`. **Fix:** Remove the sub-keywords.

- Replace `satisfy requirement` with `satisfy`
- Replace `frame concern` with `frame`

2. Multiplicity Braces

Error: `mismatched input '}' expecting {'default', '{', ';', '=', ':='}` caused by `{ ordered nonunique }`. **Fix:** Remove the braces and add a semicolon.

- Replace `{ ordered nonunique }` with `ordered nonunique;`

3. KerML Prefix

Error: Name resolution error for `KerML::CollectionFunctions::size`. **Fix:** The `KerML::` prefix is often unnecessary and causes resolution errors.

- Replace `KerML::CollectionFunctions::size` with `CollectionFunctions::size`

4. Constraint Inline Comments

Error: `extraneous input '/* comment */' expecting '}'` **Fix:** In constraint blocks, adding inline block comments `/* ... */` immediately after an expression without a semicolon breaks the parser.

- Add a semicolon to the end of the expression before the comment, or remove the comment.

5. Non-existent Imports

Error: `Package 'ProjectUnits' not found` or `Package 'RequirementTypes' not found`. **Fix:** LLMs hallucinate standard library packages. Delete the private import statements for these if they are not explicitly defined in the file.

6. State Machine Transitions ('when', 'after', custom signals)

Errors:

- `Name resolution error: couldn't resolve reference to Element 'when'`
- `Name resolution error: couldn't resolve reference to Element 'emergencySignal'`
Context: The LLM is trying to write state machine transitions based on triggers. **Question:** What is the correct standard SysML v2 syntax for state transitions triggered by signals, events, or conditions?
Answer: 'when' is not the correct keyword. Use `if` for guard conditions.

```

transition t2
  first Green
  if traffic == 0
    then Red;

```

7. Misplaced Keywords

Errors:

- **mismatched input 'Requirement' expecting {'default', '{', ';', '=', ':='}** **Context:** Sometimes the LLM capitalizes keywords or places them inappropriately. **Answer:** Ensure all structural keywords are strictly lowercase (`requirement`, `attribute`, `part`, `port`, `state`, `transition`). Capitalized versions like `Requirement` are typically used for type inheritance, but the keyword defining the element must be lowercase.

8. Misplaced Dots

Errors:

- **mismatched input '.' expecting {'default', '{', ';', '=', ':='}** **Context:** LLMs using dot-notation where it might not be supported. **Answer:** Do not use dot notation `.` for namespace qualification (e.g., importing or referencing elements in a package). Always use double colons `::` for namespaces (e.g., `ISQ::length`, `LogicalDesign::PowerPort`). Dot notation is generally reserved for traversing features within a bounded context.

9. Conjugated Ports/Tildes (~)

File: `deepseek_sysml_20260426_1a7ce4.sysml` **Line 151:** `port def ~PowerPort {` **Error:** mismatched input '~' expecting {'{', ';'} **File:** `deepseek_sysml_20260426_dca19b.sysml` **Line 324:** `private import LogicalDesign::~~PowerPort;` **Error:** no viable alternative at input 'private import LogicalDesign::~~' **Question:** What is the proper SysMLv2 syntax for conjugated ports? (It's `~portName : PortDef`, not `port def ~PortDef` or importing with `~`). **Answer:** You cannot use tildes (`~`) to denote conjugated ports in SysMLv2. You can only conjugate the usage. Preferably this is done in the context of a use. For example,

```

package ExmpleUse {
  port def PowerPort {
    in dataIn : Real;
  }
  part def PartA_Example {
    port inputPort : PowerPort;
  }
  part def PartB_Example {
    port outputPort : ~PowerPort;
  }
}

```

10. Attribute Definitions vs Types

File: `deepseek_sysml_20260426_dca19b.sysml` **Line 288:** `attribute def externalLeverDown : ScalarValues::Boolean;` **Error:** mismatched input ':' expecting '{', ';'} **Question:** You cannot specify the type of an `attribute def` with `:`. You must use `>` or `>.` **Answer:** We generally do not create a definition for an attribute. The attribute is simply added to the context it is a member of.

```
part def ExamplePart {
  attribute externalLeverDown : ScalarValues::Boolean;
}
/* Or, if added to a use */
part use examplePart1 : ExamplePart {
  attribute addedAttribute : ScalarValues::Boolean;
}
```

11. Array Indexing in Bindings

File: `deepseek_sysml_20260426_055020.sysml` **Line 752:** `binding bind flightCtrl.pwmOuts[1] = escs[1].pwmIn;` **Error:** missing '=' at '[' **Question:** How should array index elements be referenced in bindings? **Answer:** Do not include multiplicity. Just bind the elements directly.

```
binding bind flightCtrl.pwmOuts = escs.pwmIn;
```

12. Invalid Encoding/Units

File: `deepseek_sysml_20260426_1a7ce4.sysml` **Line 104:** `attribute maxSurfaceTemp : ThermodynamicTemperatureValue = 45 [C];` **Error:** token recognition error at: ' **Question:** Degree symbol broke parser. **Answer:** The degrees celcius symbol needs to be quoted. Also ensure that the SI: 🌡 package is imported.

```
attribute maxSurfaceTemp : ThermodynamicTemperatureValue = 45['°C'];
```

13. `attr` instead of `attribute`

File: `deepseek_sysml_20260426_055020.sysml` **Line 670:** `attr diameter : ISQ::length = 5 [inch];` **Error:** mismatched input 'diameter' expecting {'default', '{', ';', '=', ':='} **Question:** `attr` is not a keyword. Must use `attribute`. **Answer:** The inch symbol is not a recognized unit. the symbol "in" is the symbol for inches in the package USCustomaryUnits. You must import USCustomaryUnits in order to use it. Also ensure that the ISQBase package is imported in order to use length.

```
import USCustomaryUnits::*;
import ISQBase::*;
```

```
part def ExamplePart {  
  attribute diameter : length = 5 [in];  
}
```